Assignment 5, explained

Gradient descent, polynomials, and regularisation

A plain-language walkthrough of every question,
with the numbers from the solution notebook.

SVNIT · Department of Artificial Intelligence
Machine Learning · u24ai028

The whole assignment in one page

Assignment 4 used sklearn to fit models. This assignment asks you to see how a model actually moves toward the answer, and what happens when the model is too simple or too flexible.

- Q1–Q3 use made-up data: $y = 4 + 3x +$ noise. You walk θ toward the best line three ways: batch, stochastic, and mini-batch gradient descent.
- Q4 uses a curve, $y = 0.5\,x^2 + x + 2 +$ noise. A degree-1 line underfits. A degree-300 polynomial overfits. Degree 2 matches the truth.
- Q5–Q6 use real red-wine data. Ridge, Lasso, and Elastic Net predict the quality score. Then logistic regression predicts "good wine" (score 7 or more).
- Q7 uses the Iris flowers. Logistic regression draws the boundary between three species from petal length and petal width.

Random draws use seed 42, so a re-run gives the same θ.

The one idea behind Q1–Q3

A line is $y = \theta 0 + \theta 1\,x$. $\theta 0$ is the intercept, $\theta 1$ is the slope. The error of a guess is the mean squared error:

$$\text{MSE} = \text{average of (prediction} - \text{actual)}^2$$

Gradient descent starts at a random θ and repeatedly steps downhill on that error. The gradient is the slope of the error. The learning rate η says how big the step is.

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \times \text{gradient}$$

The data is $X = 2 \times \text{random}(100, 1)$ and $y = 4 + 3X +$ noise, so the true line is $\theta = [4, 3]$. Noise means the best fit on these 100 points is a bit off that. The Normal Equation on the same points gives $\theta 0 = 4.2151$ and $\theta 1 = 2.7701$, with MSE = 0.8066. That is the target the gradient methods are walking toward. The starting θ was random: about [0.01, 1.45].

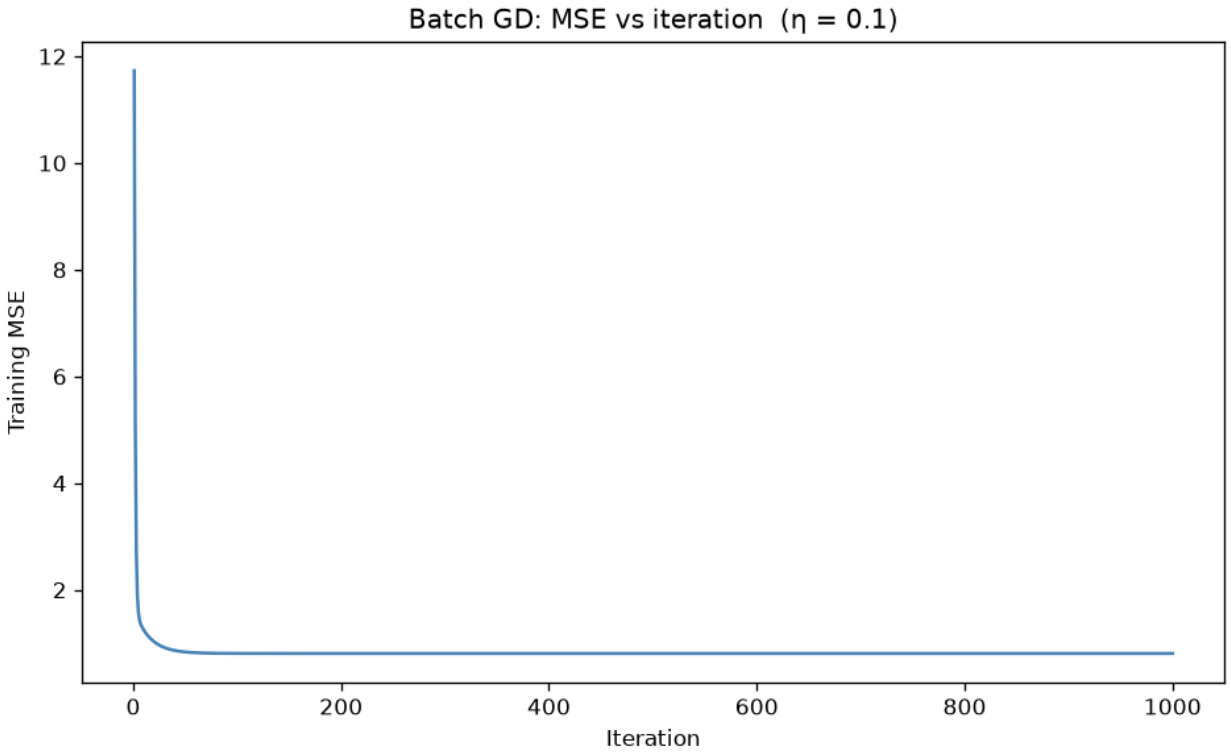
Q1. Batch gradient descent

Every step uses all 100 rows at once. Learning rate is 0.1. There are 1,000 iterations, so 1,000 parameter updates. The gradient of the MSE is:

$$\text{gradient} = (2 / m)\ X^T (X\theta - y)$$

The 2 appears because the derivative of $(\text{error})^2$ brings down a 2. m is 100. A column of 1s is glued onto X so $\theta 0$ has something to multiply.

After 1,000 steps, batch GD lands on $\theta 0 = 4.2151$ and $\theta 1 = 2.7701$. That is the same answer as the Normal Equation, so it has fully converged. Training MSE ends at 0.8066. The MSE drops below 1.0 at epoch 22. One epoch here is one pass over the data, which for batch GD is one iteration.



The error falls almost immediately, then the line is flat. Extra iterations after that do not change θ.

Say this: Batch GD uses every row on every step. With η = 0.1 it reaches the least-squares line, the same θ as the Normal Equation.

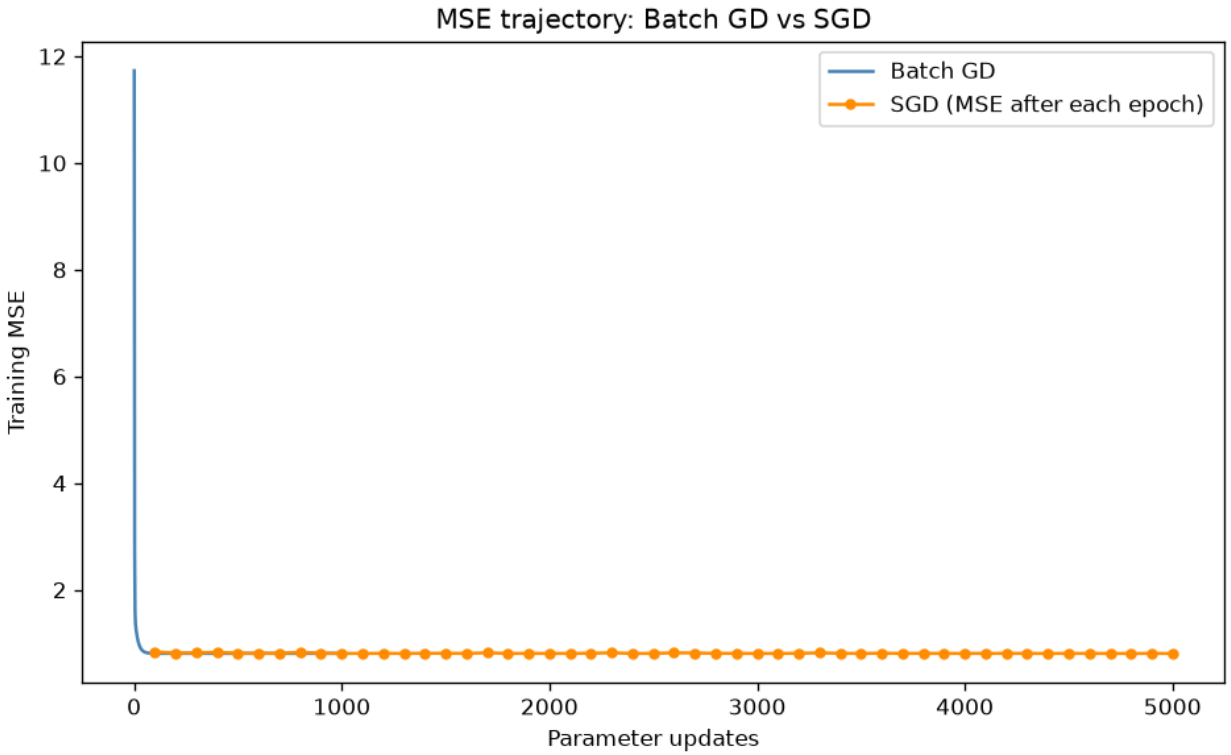
Q2. Stochastic gradient descent

Stochastic means random. Each step uses one random row, not all 100. The step size shrinks over time so the path can settle:

$$\eta = t0 / (t + t1) \quad \text{with } t0 = 5, t1 = 50$$

t starts at 0 and goes up by 1 on every row. Early on, η = 5/50 = 0.1, the same size as batch GD. Later the steps get smaller.

50 epochs × 100 rows = 5,000 updates. Batch GD only did 1,000. SGD's final θ is 4.2151 and 2.7535, and the final MSE is 0.8069. Almost the same place as batch GD, with a slightly different slope because the path is noisy. MSE is already under 1.0 after the first epoch, because that epoch contains 100 updates.



Blue is batch GD (1,000 updates). Orange dots are SGD, measured after each epoch. Both sit on the same low error. SGD just keeps taking small noisy steps out to 5,000 updates.

Say this: SGD updates once per row, 5,000 times in total. Batch GD updates once per full pass, 1,000 times. They finish at almost the same θ.

Q3. Mini-batch, and the comparison table

Mini-batch is the middle ground. Each step uses 20 rows (batch_size = 20), in random order. The question does not give a separate learning rate, so the notebook uses the same decaying schedule as SGD, η = 5 / (t + 50). 50 epochs × (100 / 20) = 250 updates.

Final mini-batch θ is 4.1349 and 2.8384. Final MSE is 0.8085. It crosses MSE < 1.0 at epoch 6.

Method	**θ0**	**θ1**	**Epochs to MSE<1**	**Updates**
Batch GD	4.2151	2.7701	22	1,000
Stochastic GD	4.2151	2.7535	1	5,000
Mini-batch GD	4.1349	2.8384	6	250

Read the "epochs" column carefully. An epoch is one pass over the 100 rows. SGD looks fastest in epochs only because each epoch hides 100 updates. Mini-batch does the fewest updates overall (250) and still lands near the same error. Batch GD takes the most epochs, but every step is exact, which is why it matches the Normal Equation digit for digit.

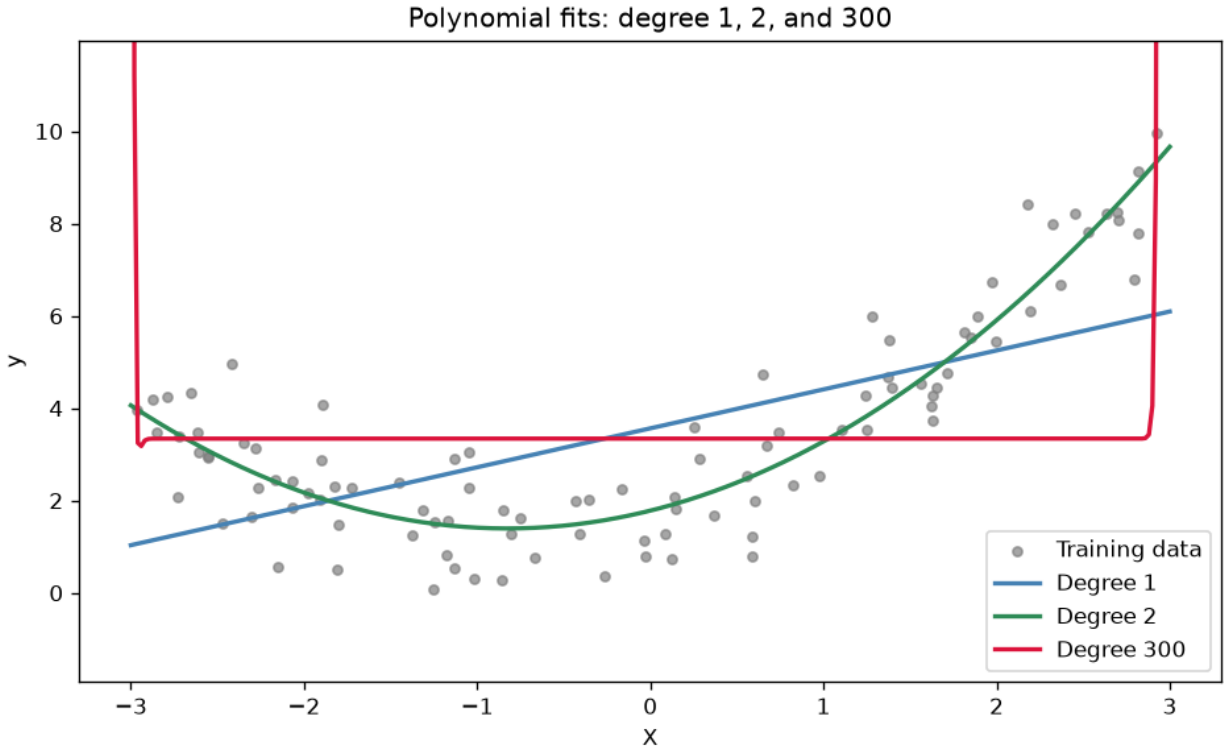
Say this: All three land near θ = [4.22, 2.77] and MSE ≈ 0.81. They differ in how many rows they use per step, and therefore in how many steps they take.

Q4. Too simple, just right, too flexible

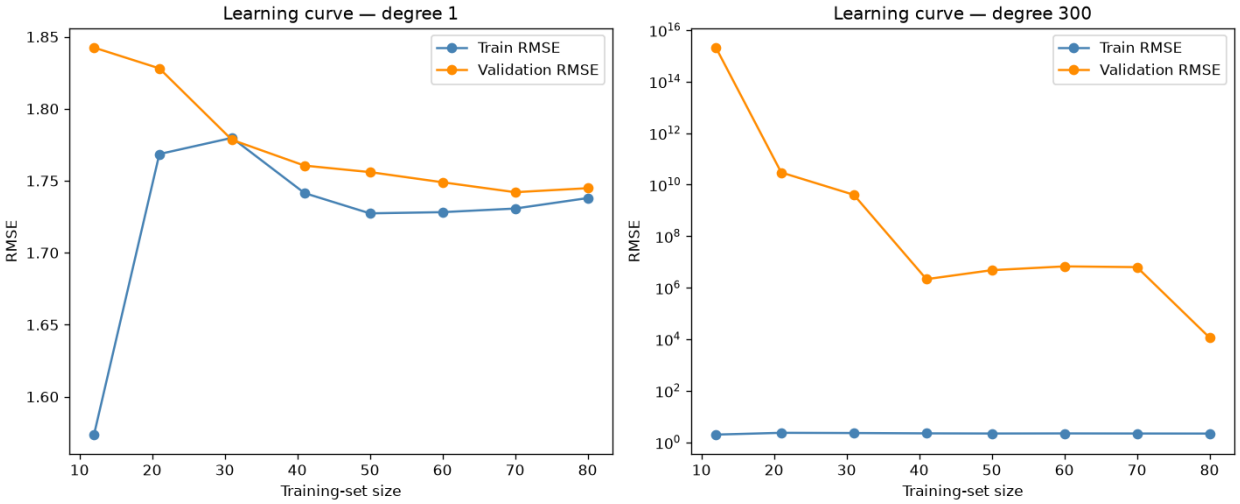
New data: 100 x-values spread from −3 to 3, and $y = 0.5\,x^2 + x + 2 +$ noise. The truth is a parabola. PolynomialFeatures builds columns x, x^2, x^3, … up to the degree you ask for. LinearRegression then fits a weight on each column.

Degree	Train RMSE	What you see
1	1.74	a straight line through a curve
2	0.88	the true shape, error ≈ the noise
300	2.20	a curve that shoots off at the edges

Degree 1 cannot bend, so it misses the dip in the middle. That is underfitting: both training and validation error stay high, and they stay close to each other (about 1.74). Degree 2 is the degree the data was built from, so the leftover error is basically the noise. Degree 300 has far more knobs than data points. On the learning curve its validation RMSE is about 11,500 while training RMSE is about 2.2. That gap is overfitting. The right-hand plot uses a log scale because the validation error is thousands of times larger than the training error.



Blue line underfits. Green curve follows the points. Red degree-300 curve is flat in the middle and explodes at $x = -3$ and $x = 3$.



Left: degree 1, train and validation error stay together and high. Right: degree 300, validation error (orange) stays far above training error.

Say this: Degree 1 underfits because a line cannot make a parabola. Degree 300 overfits because it has more coefficients than points, so it fails on new x values.

Q5. Ridge, Lasso, and Elastic Net on red wine

The file is winequality-red.csv, semicolon-separated, 1,599 wines and 11 chemical measurements. The target is quality, a score. The split is 80/20 with random_state=42: 1,279 train, 320 test. Every model is a pipeline: StandardScaler, then the regressor. Scaling matters because alpha penalises the size of the coefficients, and that penalty is only fair if the columns share a scale.

What the three penalties do

- Ridge adds α × (sum of squared coefficients). Coefficients get small, but they rarely become exactly 0. Every feature stays in the model.
- Lasso adds α × (sum of |coefficients|). Coefficients can become exactly 0, so Lasso deletes features.
- Elastic Net is both penalties at once. sklearn's default mix is l1_ratio = 0.5, half Lasso and half Ridge.

α is tried at 0.001, 0.01, 0.1, 1, and 10. Bigger α means a stronger penalty and a simpler model. The table is test RMSE. Lower is better.

Model	**0.001**	**0.01**	**0.1**	**1**	**10**
Ridge	0.6245	0.6245	0.6245	0.6245	0.6246
Lasso	0.6249	0.6269	0.6627	0.8107	0.8107
Elastic Net	0.6247	0.6263	0.6441	0.8107	0.8107

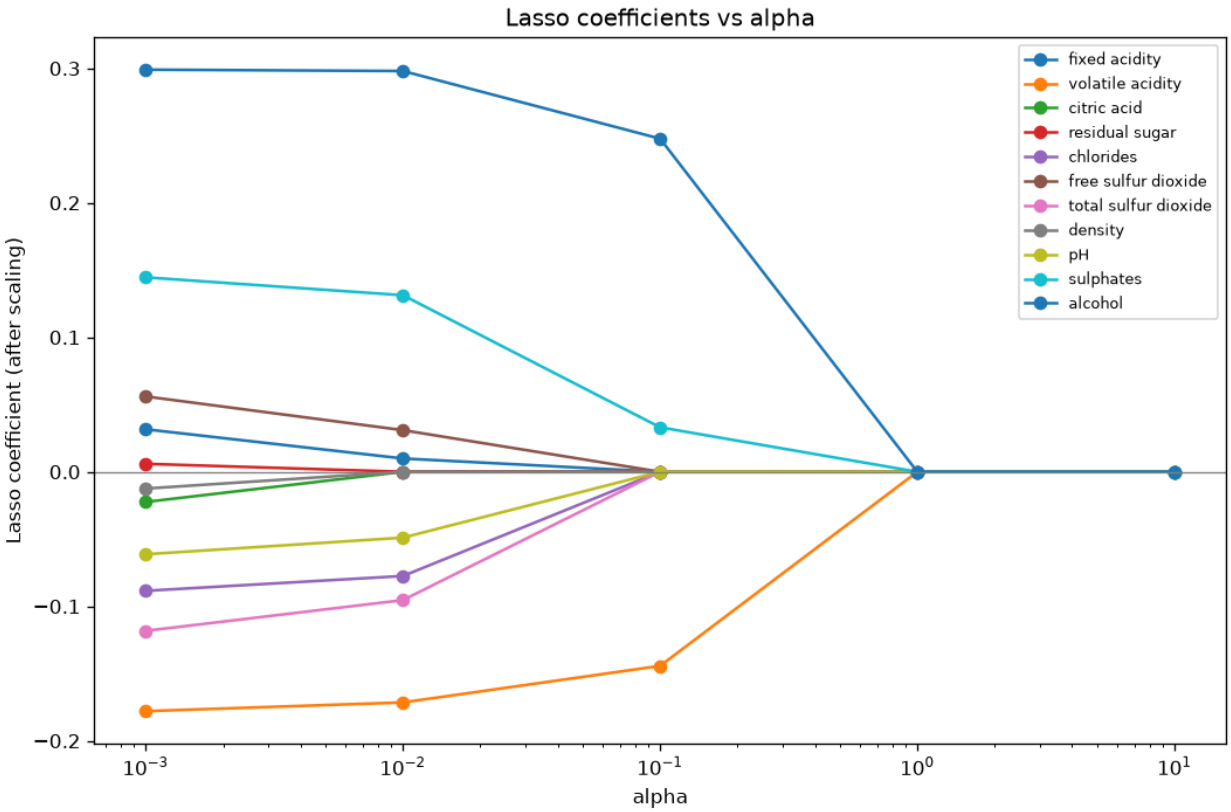
The best score in the table is Ridge at α = 0.001, RMSE 0.6245. Ridge barely changes as α grows, because shrinking coefficients a bit does not hurt this dataset. Lasso and Elastic Net get worse once α reaches 1. At α = 1 and α = 10 their RMSE is 0.8107, which is what you get from predicting the average quality for every wine: the penalty has wiped the model out.

Which Lasso coefficient hits zero first?

On the five required alphas, three features are already 0 at α = 0.01 and still nonzero at α = 0.001: citric acid, residual sugar, and density. A finer sweep between 0.001 and 0.01 separates them:

- residual sugar reaches 0 first, at about α = 0.0027
- density is next, at about α = 0.0033
- citric acid follows, at about α = 0.0057

alcohol, volatile acidity, and sulphates survive the longest. They only hit zero at α = 1. Those are the measurements the model most wants to keep.



Each line is one feature. As alpha grows, every line is pulled to 0. The lines that touch 0 earliest are the ones Lasso drops first.

Say this: Ridge is the most stable across alpha. Lasso starts deleting features as alpha grows, and residual sugar is the first coefficient to hit exactly zero.

Q6. Is the wine good? Logistic regression

quality is turned into two classes: good if the score is 7 or more, not good otherwise. There are 217 good wines and 1,382 not-good wines, so "good" is only about 14% of the data. The split is 80/20, stratified, so the test set keeps that 14%.

Logistic regression does not predict the class directly. It predicts a probability with the sigmoid function, and the usual cutoff is 0.5. Features are scaled first, same reason as Q5.

Metric	Score	Plain meaning
Accuracy	89.38%	overall fraction correct
Precision	69.57%	of wines called good, this fraction really is
Recall	37.21%	of truly good wines, this fraction was caught
F1	48.48%	balance of precision and recall

Accuracy looks strong because always saying "not good" would already be about 86% right. Recall of 37% means the model misses most of the good wines. That is the number to talk about, not the accuracy.

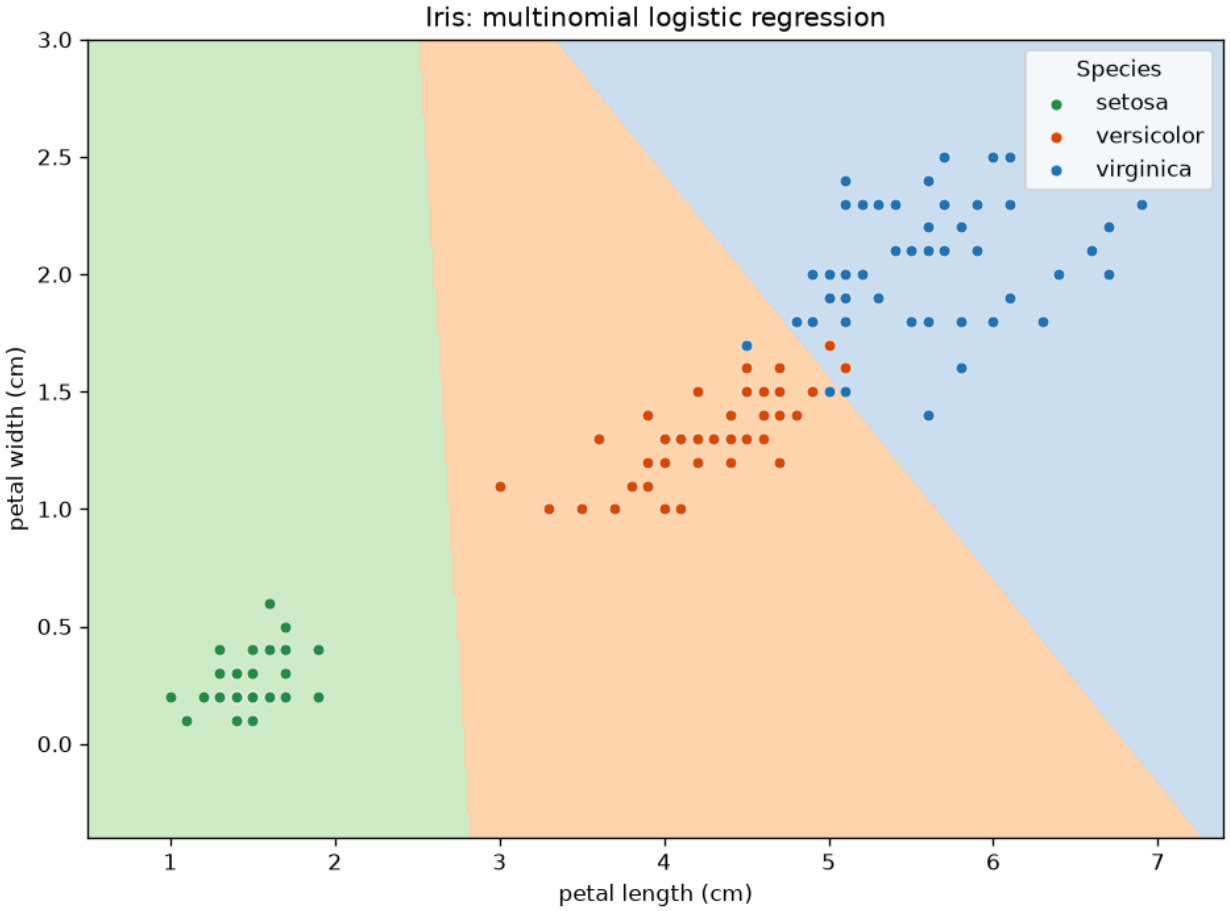
Say this: Accuracy is 89% partly because good wines are rare. Recall is only 37%, so most good wines are missed.

Q7. Iris: a picture of the decision boundary

Only two columns are used: petal length and petal width. There are three species: setosa, versicolor,

virginica. LogisticRegression on 3 classes fits the multinomial (softmax) model, which is what multi_class="multinomial" used to request. Current sklearn does that by default, and the old argument has been removed.

The filled colours are the model's prediction at every point in the plane. The boundary is where two class probabilities are equal. The dots are the 150 training flowers. Training accuracy is 96.67%. Setosa separates cleanly. Versicolor and virginica share a fuzzy border, which is where the few mistakes are.



Green is setosa, orange is versicolor, blue is virginica. Two petal measurements are enough to separate them.

Say this: With only petal length and petal width, multinomial logistic regression separates the three species at 96.7% training accuracy. Setosa is the easy one.

Short answers if you are asked

- What is a gradient? The slope of the error. We step in the opposite direction.
- What does the learning rate do? It sets the step size. Too big overshoots. Too small takes forever.
- Batch vs SGD vs mini-batch? All rows, one row, or a small group of rows, per update.
- Why does SGD's learning rate shrink? So the late steps stop bouncing around the minimum.
- Why does batch GD match the Normal Equation? Both minimise squared error, and 1,000 steps were enough to arrive.
- Underfitting vs overfitting? High error on both train and val, versus low train error and much higher val error.
- Why is degree 2 the right one? The data was built from a quadratic. Degree 1 cannot bend.

Degree 300 has more weights than points.

- Ridge vs Lasso? Ridge shrinks coefficients. Lasso can set them to exactly 0 and drop features.
- Why scale before Ridge/Lasso? Alpha penalises coefficient size. Unscaled columns would be penalised unfairly.
- Why is wine accuracy misleading? Only 14% of wines are "good", so a model can look accurate while missing most of them.